GYEOL :

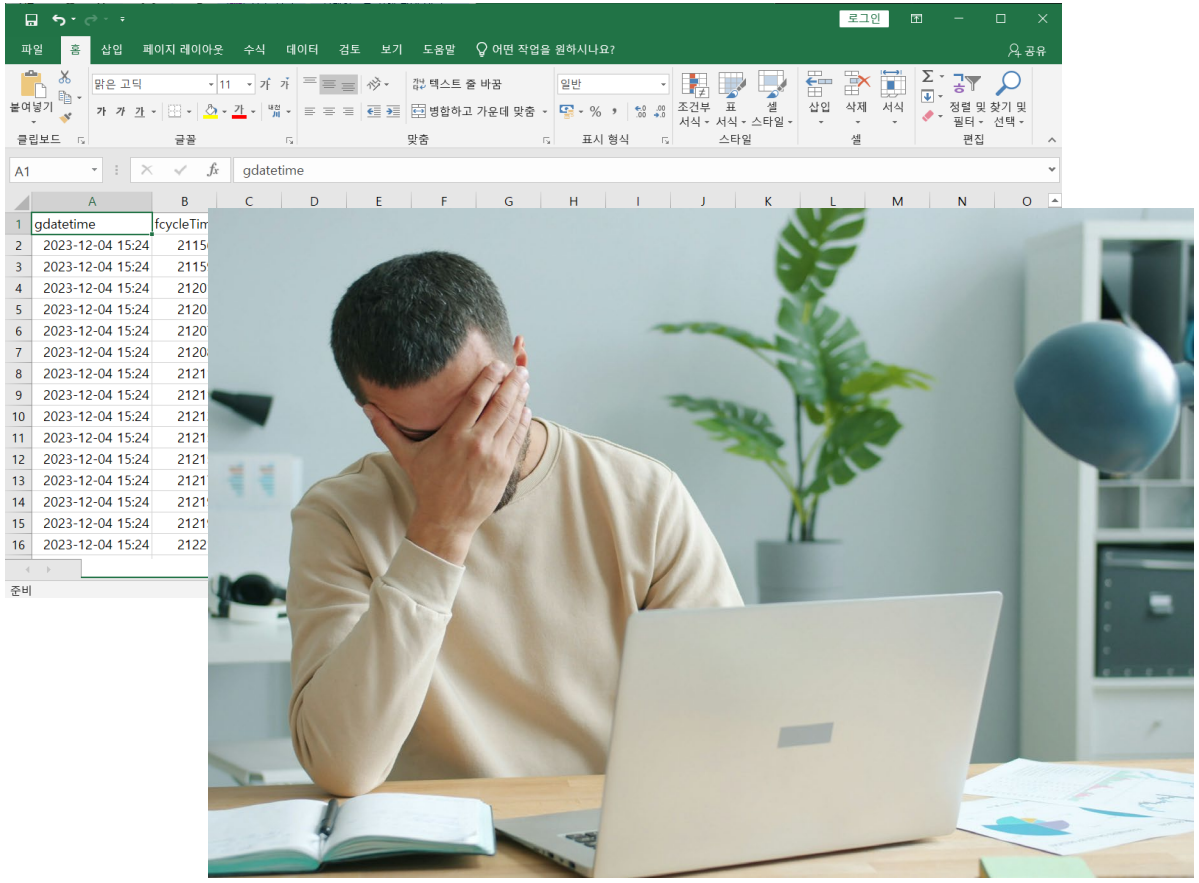
Generative Yield Engine for On-premise LLM

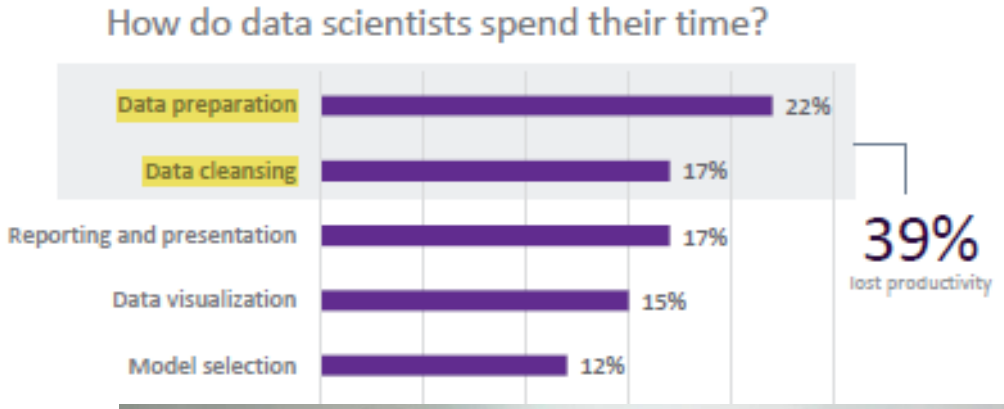
온프레미스 제조 데이터 전처리 자동화

SI 고객 대상 데이터 전처리 시스템 | 2026.05. ~ 06. | 김명선, 송병갑

데이터 전처리 자동화 필요성

"고객사 데이터 도착했다고 합니다."







왜 GYEOL인가?

기존 ETL vs GYEOL

기존 ETL — 현황과 한계

현황 SI·제조는 여전히 온프렘 ETL + 오픈소스 스택 중심

Informatica · SSIS · ODI · DataStage · Talend / Airflow + Spark

- 데이터가 외부(클라우드)로
  - 모던 스택은 클라우드 전제 · 보안 부담
- 새 공정·소스마다 새 잡·코드
  - 스키마 변경이 여러 잡에 파급 · 개발자 개입
- 안전장치(사람 승인)가 약함
  - 계보·거버넌스는 무거운 엔터프라이즈에만
- 고가 · 운영 부담
  - 가파른 학습곡선 · 오픈소스는 직접 운영

GYEOL — 3대 차별점

데이터 보안
데이터가 공장 밖으로 안 나간다

로컬 LLM · 외부 API 0

재사용 경제성
공정이 늘어도 코드는 안 늘어난다

모달리티 단위 분할

안전한 자동화
위험 작업은 사람이 승인

3단계 권한 · 이력 추적

보안 · 확장성 · 신뢰성 — SI 도입에서 까다로운 세 가지 사항을 설계 원칙으로

데이터에 손대는 '권한'을 3단계로 나눈다

"이 작업이 데이터를 바꾸는가? 되돌릴 수 있는가?" → 사람의 개입 깊이가 달라진다.

읽기 (살펴보기)
권한 1 단계 · 자동

데이터에 무슨 일?
보기만 한다 — 절대 안 바꿈

현장 예시
컬럼·타입 확인 · 인코딩(cp949) 감지
기초 통계 · 제약 위반 점검

누가 결정?
자동 즉시 실행 (사람 개입 0)

위험도: 안전 ●○○
≈ 문서를 '읽기 전용'으로 여는 것

수정 (보정)
권한 2 단계 · 1-click 승인

데이터에 무슨 일?
바꾼다 — 그러나 되돌릴 수 있음

현장 예시
결측값 채우기 · 이상치 처리
클래스 불균형 보정

누가 결정?
사람이 미리보기 후 1-click 승인

위험도: 주의 ●●○
≈ 고친 뒤 '되돌리기' 되는 편집

삭제 · 재정의
권한 3 단계 · 승인+백업

데이터에 무슨 일?
되돌리기 어려운 변경

현장 예시
컬럼 삭제
라벨(양/불) 재정의

누가 결정?
명시 승인 + 자동 백업 강제

위험도: 위험 ●●●
≈ 지우되 반드시 '복원본'을 남김

안전한 일은 AI가 끝까지 자동으로 · 위험한 일은 반드시 사람이 승인 **⇒ 자동화의 속도와 통제를 동시에**

그래서 AI가 어떻게 '안전하게' 결과를 내는가



그래서 AI가 어떻게 '안전하게' 결과를 내는가

Backend

진단	제안	등급 분류	승인 게이트	실행	기록
AI가 데이터를 읽음 (읽기 = 자동)	'이렇게 고치자' AI는 제안만	작업마다 자동 부여 읽기/수정/삭제	수정·삭제는 멈춤 → 사람 승인	결정론 코드가 변형 AI 손 안 댐	lineage에 전부 되돌리기 가능

Gemma 4

사람의 승인 절차

안전한 작업 (읽기 = L1)

코드가 곧장 실행 →

```
status: "done", lineage_recorded: true
```

데이터 처리 완료 · 사람 개입 0

위험한 작업 (수정·삭제 = L2/L3)

코드가 스스로 멈춤 →

```
status: "awaiting_approval"
```

사람이 승인하기 전엔 데이터 0 변경

AI는 '무엇을 할지' 판단만 — '실제로 바꾸는 일'은 검증된 코드 + 사람 승인 | AI가 틀려도(환각) 원본 데이터는 안전

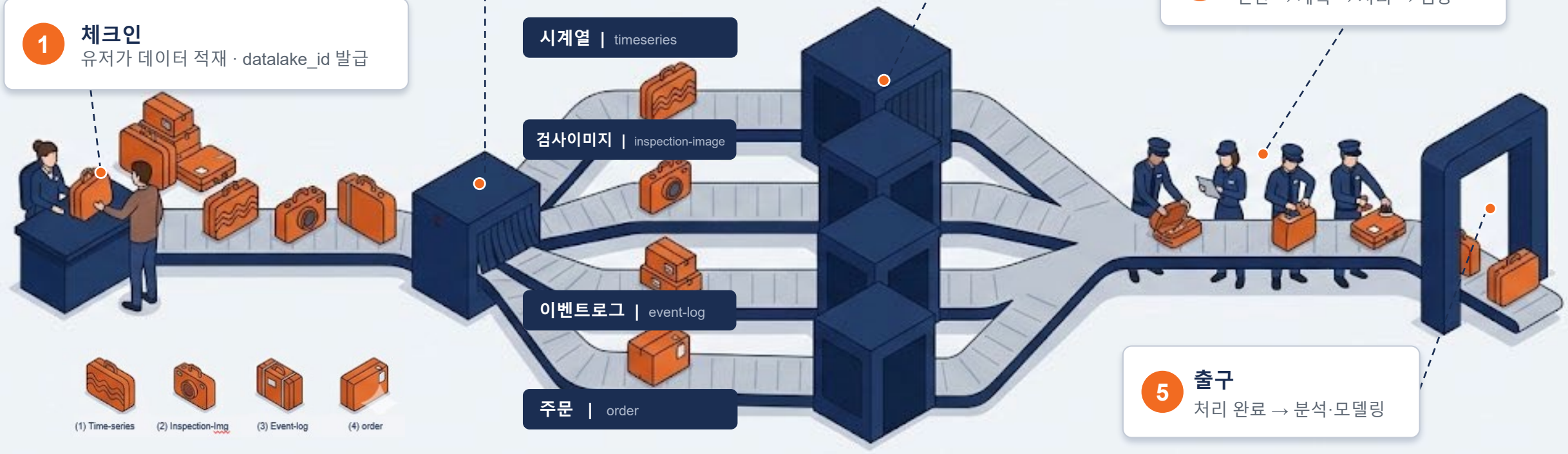
공항 수하물과 같은 — Data Pipeline

1 체크인
유저가 데이터 적재 · datalake_id 발급

2 짐 분류
모달리티 4종 자동 분류

3 전용 검색대
MCP 서버 4종 · :8101~8104

4 에이전틱 처리
진단 → 계획 → 처리 → 검증



5 출구
처리 완료 → 분석·모델링

POINT ! 짐은 '종류(모달리티)'로 분류 → 통과하는 짐 분류(모달리티) & 검색대(MCP) 유지 / 추가 용이

왜 '데이터 종류'로 나눴나 - modality

KeyPoint: 데이터 '생김새'는 공정(데이터셋)마다 달라도, 전처리 '작업의 유형'은 일정하다.

✗ 데이터별 전처리

- 도장 · 주조 · MCT · 검사… 공정마다 따로
- 도구 7종 + 권한 3등급을 매번 새로 정의
- 새 공정 = 새 코드 → N배 개발 · 유지비
- 규칙이 제각각 → 감사 · 일관성 무너짐

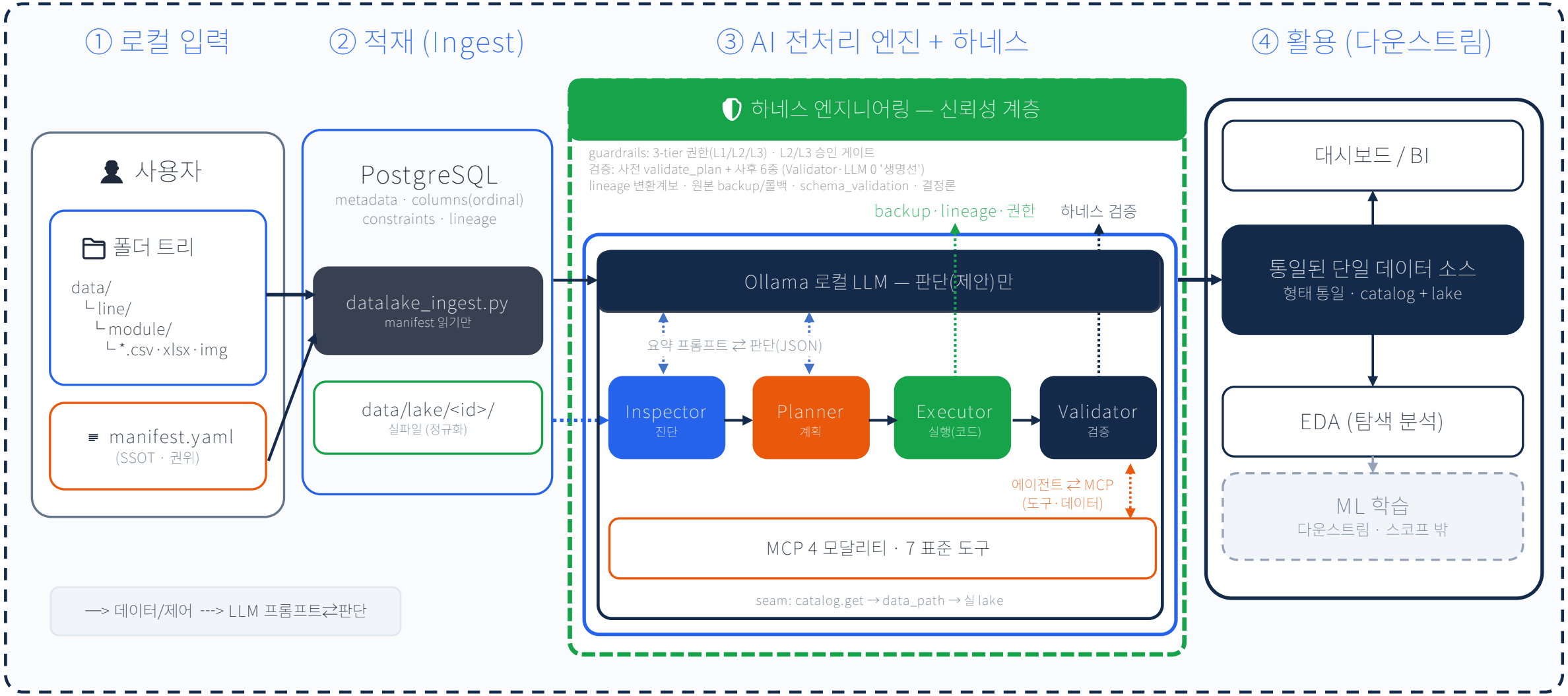
✓ 데이터 종류(모달리티) + 권한(승인)

- 데이터 패턴 4종으로 수렴 (시계열 · 이미지 · 이벤트 · 주문)
- 7 도구 × 3 권한 = 한 벌만 정의
- 4 모달리티가 그대로 공유
- 새 공정이 와도 도구 · 권한 0 추가
  — 데이터만 추가로 연결

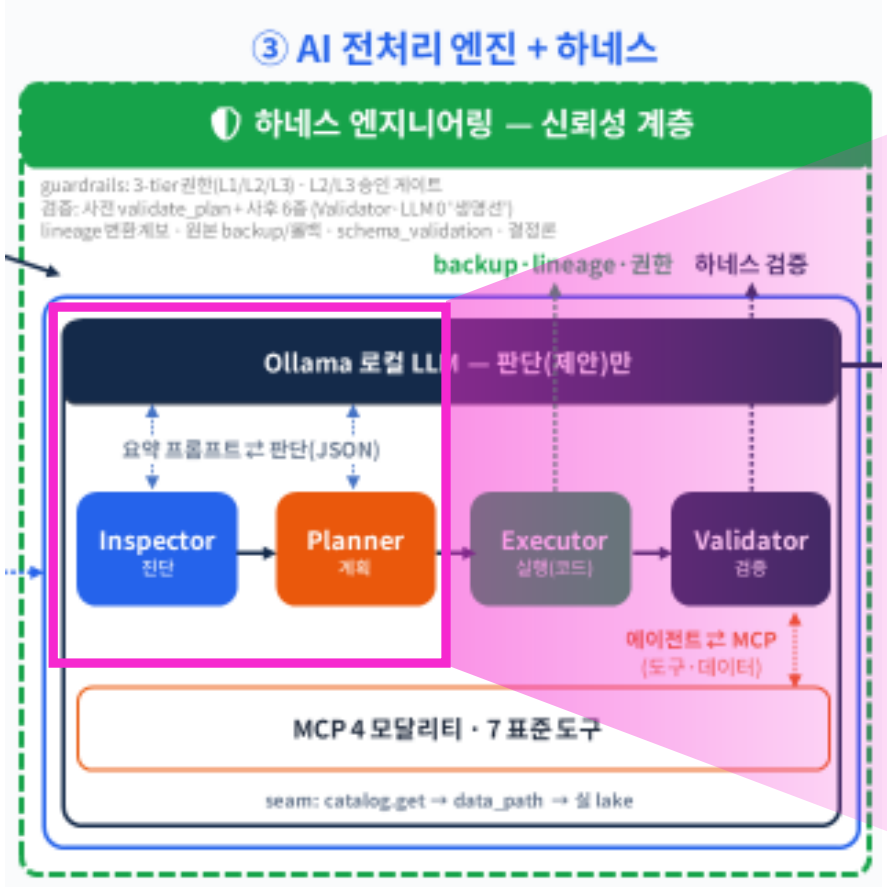
라인 · 공정을 늘려도 개발비가 안 는다 — '데이터 종류'는 유한하지만 '공정'은 무한하기 때문.

데이터 · 프롬프트 흐름 + 하네스 엔지니어링 (신뢰성 계층)

온프레미스 / 폐쇄망 · Docker Compose — 데이터 외부 유출 0 (외부 노출 = 웹 대시보드 HTTP 뿐)



실전 검증: AI가 실패해도 시스템이 동작



LLM INTERPRETATION (GEMMA)

```
{
  "_llm_error": "",
  "_hint": "ollama 컨테이너와 모델(pull) 확인"
}
```

model 호출 실패

Inspector / Planner
정상 동작

PREPROCESSING PLAN · PLANNER (2 STEPS)

규칙 기반 계획 (LLM 보정 실패)

⚠ 이 계획에는 승인이 필요한 단계(L2/L3)가 포함되어 있습니다. Executor 실행 전 사용자 승인 필요.

1	L2	clean_masking · CNC_ServoLoad_1	'CNC_ServoLoad_1'에 마스킹 문자/혼재 dtype. NaN 변환 후 수치화 필요.
2	L1	compute_stats	전처리 전후 비교를 위한 기초 통계.

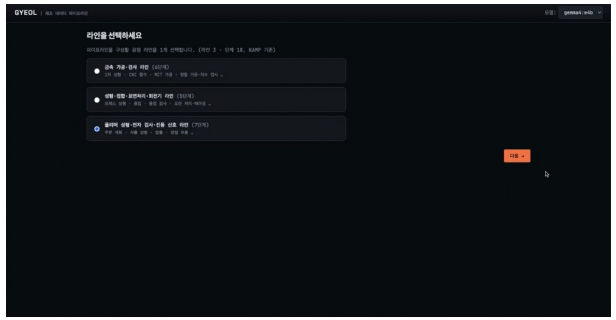
실전 검증: 8가지 '더러운 데이터' 챌린지

1. CP949 한글 인코딩
   → 인코딩 자동 감지·재해석
2. 헤더 없는 CSV
   → 헤더 부재 감지·재파싱 제안
3. 마스킹·혼합 타입 (*, **)
   → 마스킹 제거 후 숫자 복원
4. 극단적 클래스 불균형
   → 균형 전략 4종 제시 (97:3 등)
5. 멀티시트 엑셀
   → LOT 기준 병합
6. LOT 메타 결측행
   → 결측 감지·보정/제거 제안
7. 이미지 해상도/모드 불일치
   → 표준화 (RGB/RGBA·크기)
8. 혼합 라벨링 방식
   → 라벨 출처 통일

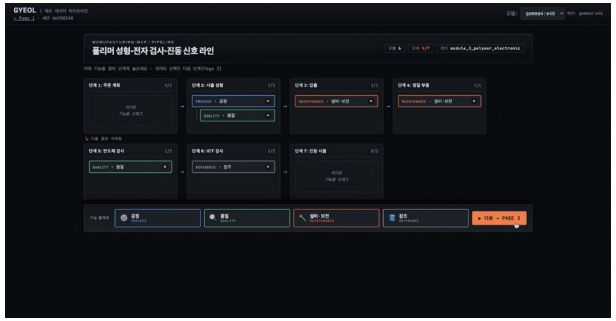
현장 데이터를 일부러 심어 '진단 → 대응 제안 → 처리'를 검증

동작 흐름: 6단계 웹 UI

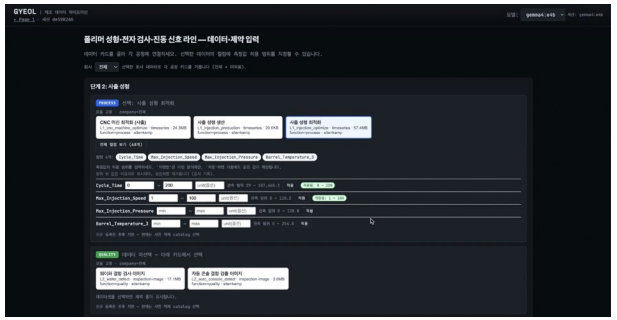
1. 라인 선택
다룰 제조 라인 선택



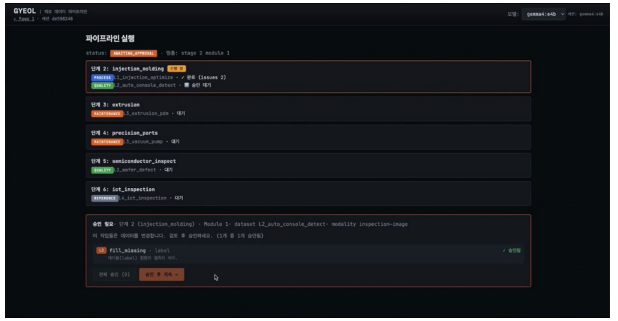
2. 파이프라인 구성
공정 노드 단계별 배치



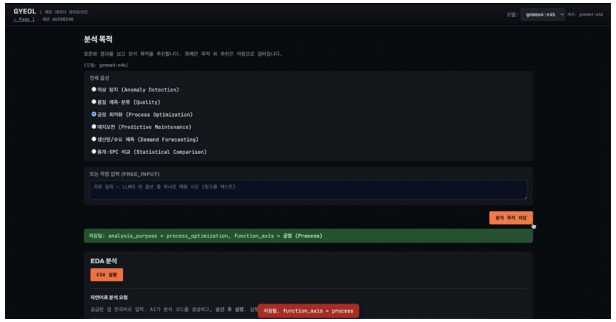
3. 데이터·제약 입력
데이터셋 연결 + 정상 범위 제약



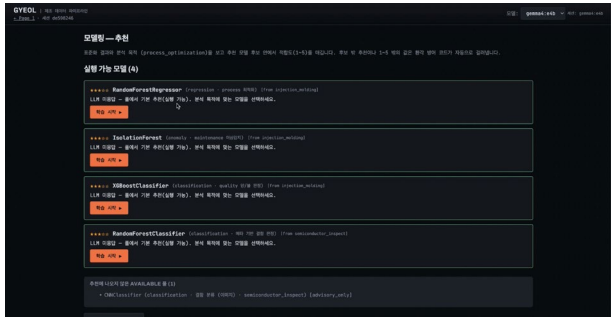
4. 실행·승인
전처리 실행, L2/L3 옵션 카드 승인



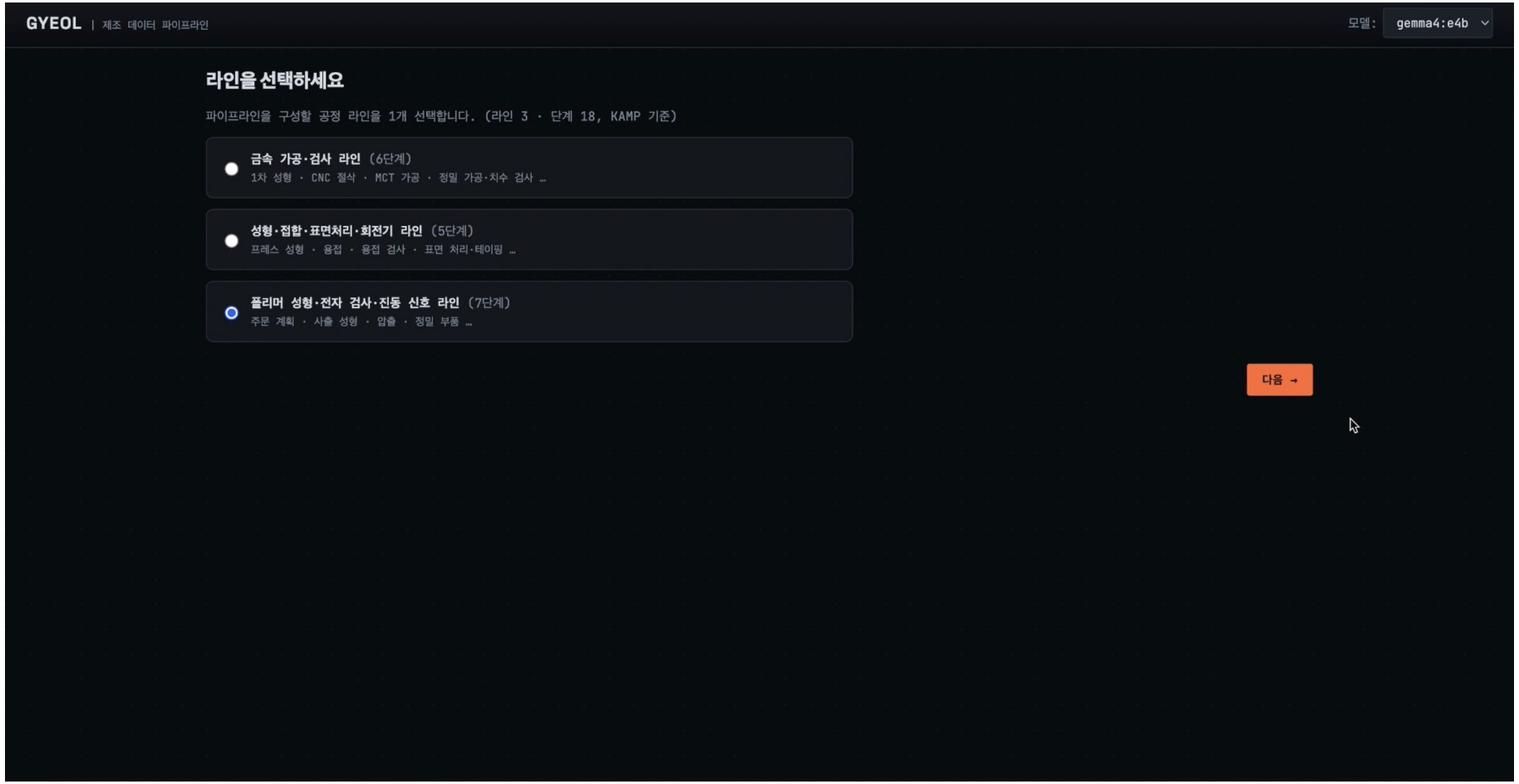
5. EDA(탐색 분석)
분석 목적 추천 + 차트



6. 모델링
데이터에 맞는 모델 추천



동작 흐름: 6단계 웹 UI - **시연 영상**



읽기 전용 코드 감사 (a3c9f25, 두 워크트리) + 런타임 관측

데이터 주권 검증 — 정적 점검 + 런타임 1차 실측

✓ 정적 점검 — 외부 유출 경로 0건

확인 **텔레메트리 라이브러리 0건**
requirements · package.json 전수 — HuggingFace · ChromaDB · PostHog · Sentry 없음

확인 **HuggingFace 허브 다운로드 0건**
코드 grep — hub 호출 없음 (매치 1건은 execute_pipeline 오탐)

확인 **sklearn 원격 fetch 0건**
fetch_openml 등 없음 — 로컬 KAMP로만 학습

확인 **HTTP 호출 대상 = 전부 내부**
httpx/fetch/urllib 전부 ollama · mcp · backend(로컬/도커)만

확인 **코드 내 URL 11개 점검**
9개 내부 + 2개 무해 (json-schema 식별자 · httpx 테스트 호스트)

확인 **Ollama · MCP 엔드포인트 = 내부**
ollama:11434 · mcp-* :8101–8104 — 외부 도메인 없음

✓ 런타임 관측 — 스택 외부 egress 0

- 컨테이너 8개 netns(/proc/net/tcp) 직접 측정
- 호스트 노이즈 배제 — 스택만 분리 관측
- idle 베이스라인 + e4b 추론 1회 · 55초/21회
- ollama · backend · MCP×4 · postgres · frontend 모두 0

측정 방식 (추측 없이 raw 기반)

✓ 정적: 읽기전용 grep · a3c9f25 · 두 워크트리 동일
✓ 런타임: byeonggab89 호스트 · 컨테이너 netns 격리
(호스트 ss는 노이즈로 미사용)

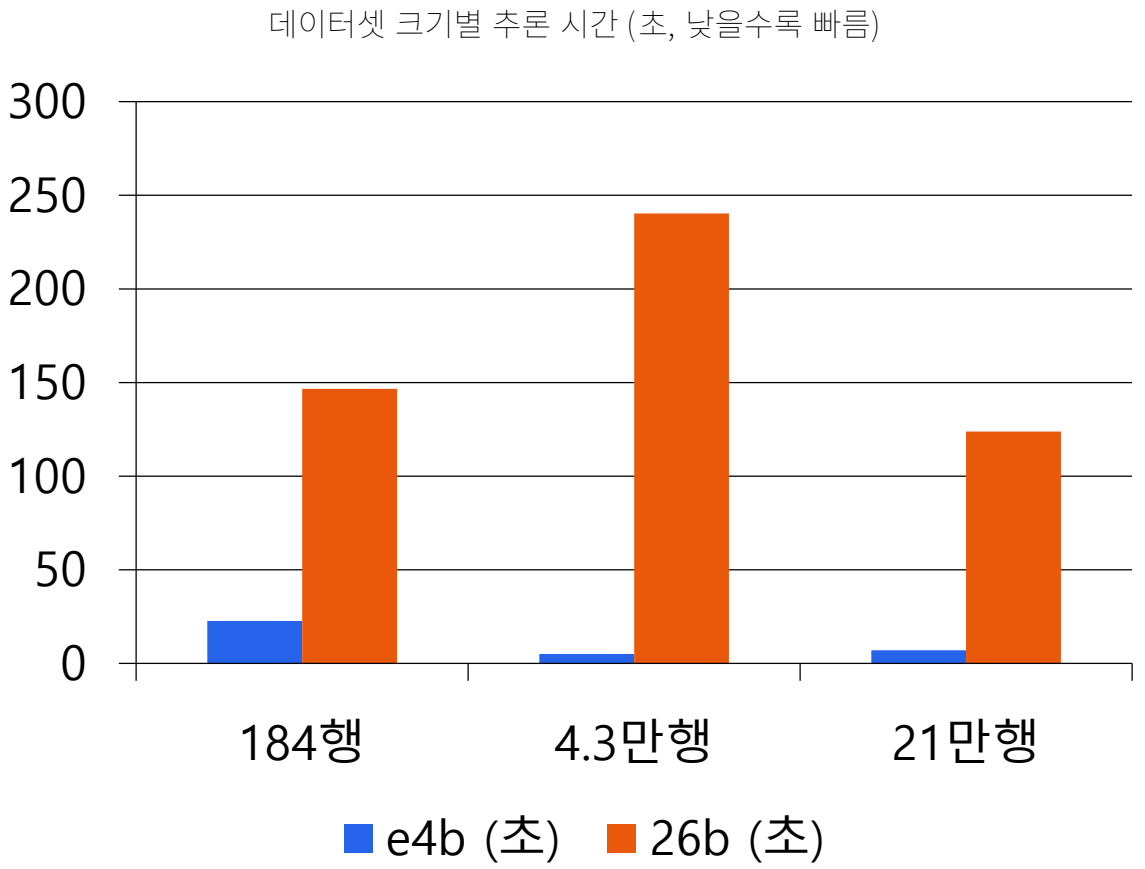
결론 – 무손실 확정

✓ 정적 : 데이터 유출 코드 · 의존성 0건 확정
✓ 런타임 : 추론 중 스택 외부 egress 0 관측

성능 실증: 보급형 GPU에서 동작

모델	VRAM	검사 시간
gemma4:e4b (개발·경량)	~3.5 GB	5–23초
gemma4:26b (운영·정밀)	~7.3 GB	약 2–4분

- 테스트: RTX 3070 (VRAM 8GB) — 보급형 단일 GPU
- 워밍업 효과: e4b 반복 추론 시 최대 56% 단축
- 31B Dense는 8GB에서 불가 → e4b/26b로 고정
- 고가 GPU 없이 PoC 가능, 정밀 운영은 GPU 증설



출처: scripts/measure_results_warm.csv (실 데이터레이크 3종, warm 실측)

정직한 현황

현재 구현 상태 & 로드맵

☑ 동작 · 검증 완료

- 4단 에이전트 파이프라인
- 4개 모달리티 전부 end-to-end 연결
  (시계열 · 주문 · 이벤트로그 · 이미지)
- 실데이터 seam: catalog(PG)→실 lake
  → 진단~검증 전 과정 관통
- 4개 MCP × 7종 도구 (동일 계약)
- 8챌린지 + 3단계 권한 + lineage
- 6단계 웹 UI

🎯 스코프 경계 (의도된 설계)

- 산출물 = 신뢰 가능한
- 통일된 단일 데이터 소스
- ML · 대시보드 = 다운스트림 소비자
  (코어 스코프 밖)
- ⇒ ML 미구현 = 결함 아님
  (의도된 경계 · 완성도 영향 0)

🗺 남은 일 (하드닝 · 로드맵)

- 프로덕션 배포 형태
  (MCP 모달리티별 컨테이너+lake)
- 현재 단일 호스트 smoke 관통 검증
- 실제 MCP 프로토콜 전환
  (동일 계약→무손실 교체)
- RAG · 공정 통합 분석

코어 산출물은 '신뢰 가능한 통일 데이터 소스'까지. ML · 대시보드는 그 위에서 자유롭게 — 그게 신뢰성의 핵심.

비즈니스 가치 & 다음 단계

도입 효과

- 전처리 리드타임 단축 (수작업 → 클릭+승인)
- 데이터 주권 확보 (온프레미스, 외부 전송 0)
- 공정 확장 시 추가 개발비 최소화 (재사용)
- 감사·복구 가능한 안전한 자동화

제안 — 단계적 도입

1단계 · PoC

실데이터 1개 라인 / 보급형 GPU 1장

2단계 · 확장

모달리티 확장 + 운영 GPU 적용

3단계 · 통합

공정 통합 분석 · RAG 확대

사람이 코드를 짜는 ETL에서, AI가 제안하고 사람이 승인하는 전처리로.

온프레미스 제조 데이터 전처리 자동화 — 데이터 처리 흐름 & 기술 스택

mfg-mcp · 기술 스택

로컬 100% · 외부 API 0
모든 추론·로깅·데이터 외부 미유출

데이터 처리 흐름

USER

웹 브라우저

HTTP · HTTPS

FRONTEND

6패널 마법사 UI

- React 18
- Vite 5
- Recharts 3

BACKEND

API 오케스트레이터

- FastAPI
- Uvicorn
- Pydantic v2 · httpx

AGENT 파이프라인 · Python

SDK · LangGraph 미사용 (직접 구현)

1 Inspector > 2 Planner > 3 Executor > 4 Validator

+ Context Aggregator
· Harness 가드레일 (권한 L1 자동 / L2 승인 / L3 차단·백업)
· Lineage 기록

EDA · ML 엔진

EDA pandas · numpy (FFT)
AST 코드 샌드박스
ML scikit-learn · XGBoost
imbalanced-learn ·
seed 42

에이전트 계층이 호출·조회하는 자원

MCP SERVERS · 모달리티 4종

각각 독립 FastAPI 컨테이너 · 표준 도구 7종 동일 계약

- timeseries · 8101 — 설비 시계열 · FFT
- inspection-image · 8102 — 검사 이미지 · 결함
- event-log · 8103 — LOT 이벤트
- order · 8104 — 주문 · 생산량

LLM 추론 엔진

Ollama
gemma4 e4b · 26b

로컬 전용 · 외부 API 0 · 8GB VRAM

데이터 저장

PostgreSQL 16
+ pgvector · asyncpg

Parquet (pyarrow) ·
스키마 metadata / lineage / agent_io 보관

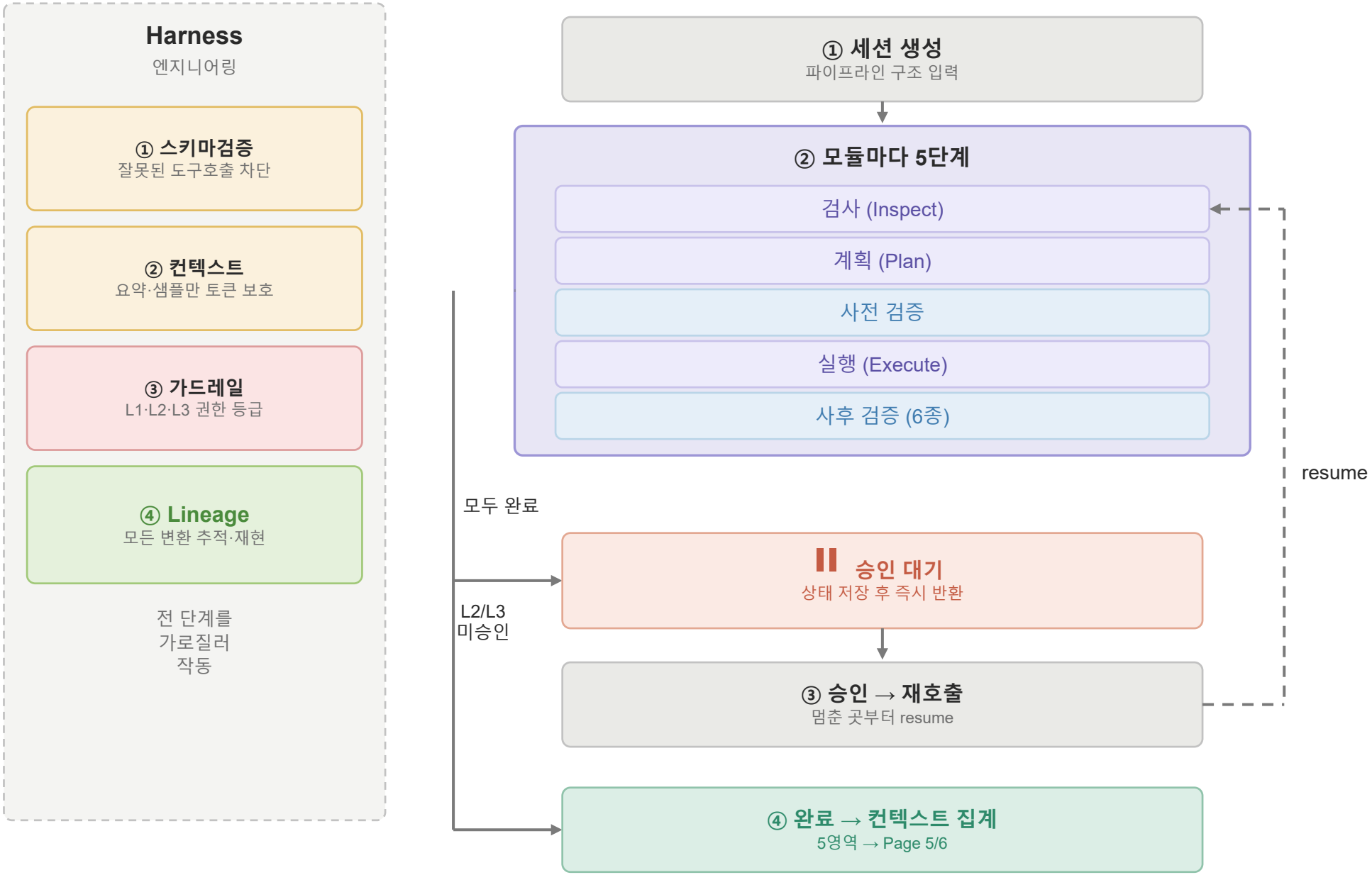
INFRA Docker Compose (7 서비스) · 볼륨 마운트
· NVIDIA Container Toolkit (GPU)

QA pytest · pytest-asyncio
· httpx ASGITransport (FastAPI 인프로세스 테스트)

scripts 전용 도구 (런타임 별도): matplotlib — 발표 PPT · 벤치마크 이미지 생성용 (requirements 미편 · 런타임 미설치 · EDA 엔진은 미사용)

감사합니다.

파이프라인 흐름 (가운데) + Harness 안전 계층 (양옆)



Harness는 한 곳이 아니라 흐름 전체에 작동: 도구호출(①)·LLM입력(②)·실행권한(③)·기록(④)